Soluzione A: unico ambiente sicuro

❶ Appena avviato Spyder andare in

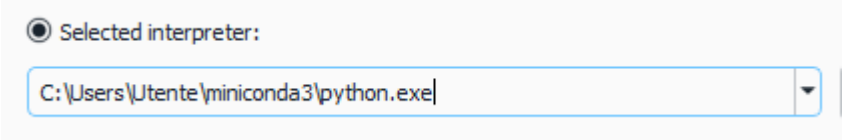
Tools ⟶ Preferences ⟶ Python Interpreter

e scegliere quello di Anaconda3 o Miniconda3, cioè qualcosa di questo tipo:

C:\Users\Utente\miniconda3\python.exe

C:\Users\Utente\miniconda3\python.exe

ad esempio così:



👉 Il vantaggio principale è usare **un Python più standard e controllabile** rispetto al Python interno di Spyder.
Ad esempio, scegliendo un python come i due sopra indicati (cioè di anaconda o miniconda), si può gestirlo meglio da CMD con comandi di questo tipo:

```
python -m pip install dmba
python -m pip list
python -m pip uninstall dmba
```

👉 Il Python interno di Spyder, invece, è più **un Python "di servizio"**: serve soprattutto a far funzionare Spyder. Modificarlo troppo con pacchetti utente non è ideale.

Quindi:

Scelta	Vantaggio	Svantaggio
Python interno di Spyder	già pronto, semplice, nessuna configurazione	meno pulito da modificare; è legato a Spyder
Python esterno standard	più controllabile da CMD e pip; più adatto al codice utente	senza venv/conda rimane comunque un ambiente unico, quindi meno isolato

👉 In altri termini, scegliere un Python esterno (a Spyder) serve soprattutto a **non sporcare il Python interno di Spyder**.

❷ Se al passo ❶ avete cambiato l'interprete - ed in generale ogni qualvolta si cambia l'interprete python, anche se appena aperto Spyder - è buona norma **riavviare il kernel**, in questo modo:

Consoles ⟶ Restart kernel

o, ancora meglio: chiudere e riaprire Spyder.

⚠️ Motivo: la console IPython già aperta continua spesso a usare il **vecchio kernel** finché non viene riavviata.

❸ Verificare quale interprete Python si sta usando in Spyder.

In teoria (quasi sempre) esso è quello visibile in *Tools ⟶ Preferences ⟶ Python Interpreter*.

Ma la verifica più sicura è la seguente (dalla consolle di Spyder):

```
import sys
print(sys.executable); print(sys.version)
```

Ottenete qualcosa di questo tipo (se non avete cambiato il python interno):

```
C:\Users\Utente\AppData\Local\spyder-6\python.exe
3.11.13 | packaged by conda-forge | (main, Jun  4 2025, 14:39:58) [MSC v.1943 64 bit (AMD64)]
```

altrimenti il python che avete selezionato voi.

⭐ Si noti che il Python di Spyder è il 3.11! ⭐

❹ Installare i package in questo modo:

Soluzione	Valutazione
`conda install ...` da CMD	🔴 bloccato / non disponibile
`%pip install ...` da console IPython Spyder	🟠 possibile, ma Spyder lo sconsiglia
`pip install ...` da console IPython Spyder	🔴 da evitare
`python.exe -m pip install ...` da CMD	🟢 soluzione consigliata

```
In [12]: %pip install dmba

Installing packages through the IPython console doesn't work reliably in Spyder. Please use a system terminal to do that, i.e.
cmd.exe on Windows, Terminal on macOS or xterm on Linux.
```

❺ La soluzione indicata al passo ❹ deve essere scritta, in modo più completo e preciso, come:

```
set PYTHONNOUSERSITE=1
"C:\Users\Utente\AppData\Local\spyder-6\python.exe" -c "import site;
print(site.ENABLE_USER_SITE)"
```

Deve dare False.

E poi:

```
"C:\Users\Utente\AppData\Local\spyder-6\python.exe" -m pip install dmba
```

Ovvero il cammino del Python usato da Spyder (se non modificato).

Soluzione B: venv come gestore di ambienti alternativo a conda

Usare *venv* come gestore di ambienti (anziché conda).

venv è **il sistema standard di Python per creare ambienti virtuali isolati**. Un ambiente *venv* contiene un **proprio interprete Python, un proprio pip e una propria cartella di package installati**, separata dagli altri progetti e dal Python principale del sistema. Questo permette di **evitare conflitti** tra versioni diverse delle librerie e di **non "sporcare" l'installazione globale di Python**. *venv* non installa direttamente i package: per questo si usa normalmente **pip all'interno dell'ambiente virtuale**. L'attivazione del *venv* **modifica temporaneamente il PATH della sessione terminale**, facendo puntare i comandi python e pip all'ambiente attivo. A differenza di conda, *venv* **non esegue dependency solving globale** e non gestisce in modo avanzato librerie native o canali di distribuzione. Essendo incluso nella libreria standard di Python, è leggero, semplice e spesso molto utile in ambienti aziendali dove conda è bloccato o non disponibile.

Si voglia gestire un VE di nome "venv-python".

Ecco i comandi principali da cmd (non da spyder).

NB. Utilizzare il python selezionato in spyder.

Ordine	Operazione	Comando	Note
1	Verificare che `venv` sia disponibile	`"C:\Users\Utente\AppData\Local\spyder-6\python.exe" -m venv --help`	Se compare l'help, `venv` è disponibile
2	Creare il venv	`"C:\Users\Utente\AppData\Local\spyder-6\python.exe" -m venv "%USERPROFILE%\venvs\venv-python"`	Crea l'ambiente nella cartella scelta
3	Attivare il venv	`"%USERPROFILE%\venvs\venv-python\Scripts\activate"`	Dopo l'attivazione appare il prefisso `(venv-python)`
4	Verificare il Python usato	`python -c "import sys; print(sys.executable)"`	Deve puntare a `...\venv-python\Scripts\python.exe`
5	Vedere cosa contiene il venv appena creato	`python -m pip list`	Mostra i package iniziali, di solito `pip` ed eventualmente `setuptools`
6	Aggiornare pip	`python -m pip install --upgrade pip`	Meglio farlo subito dopo la creazione
7	Installare un package	`python -m pip install openpyxl`	Installa dentro il venv attivo
8	Verificare la versione installata	`python -c "import openpyxl; print(openpyxl.__version__)"`	Verifica reale tramite import
9	Vedere dettagli del package	`python -m pip show openpyxl`	Mostra versione, dipendenze e cartella di installazione
10	Disinstallare un package	`python -m pip uninstall openpyxl`	Aggiungere `-y` per evitare conferma
11	Salvare/exportare l'ambiente	`python -m pip freeze > requirements.txt`	Salva i package Python installati
12	Disattivare il venv	`deactivate`	Esce dall'ambiente virtuale
13	Lista venv "tipo conda env list"	`dir "%USERPROFILE%\venvs"`	Funziona se tutti i venv sono raccolti in quella cartella
14	Cercare venv tramite `pyvenv.cfg`	`dir /s /b "%USERPROFILE%\venvs\pyvenv.cfg"`	`venv` non ha un registro centrale come conda
15	Ricreare un ambiente dopo export	`python -m venv "%USERPROFILE%\venvs\venv-python-2"`	Prima crea un nuovo venv
16	Attivare il nuovo ambiente	`"%USERPROFILE%\venvs\venv-python-2\Scripts\activate"`	Poi installa dal file esportato
17	Reinstallare da export	`python -m pip install -r requirements.txt`	Ricrea i package salvati con `pip freeze`
18	Cancellare un venv	`rmdir /s /q "%USERPROFILE%\venvs\venv-python"`	Da fare solo dopo `deactivate`; cancella la cartella

venv **non ha un registro centrale** degli ambienti, quindi non esiste un equivalente diretto di:

```
conda env list
```

Con *venv*, gli ambienti sono semplicemente **cartelle.**

Un *venv* appena creato non è completamente vuoto. Di solito contiene almeno: *pip*, *setuptool*, *wheel* ed altri package importanti (*pandas*, *numpy*, *scipy*, *scikit-learn*).